

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 611 — Operating Systems

Abstraction, arbitration, and isolation — built by writing a kernel rather than reading about one

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 2 (with CSCE 645 and CSCE 606)

Prerequisites: CSCE 614 Computer Architecture; C proficiency; comfort with pointers and manual memory

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A working kernel: context switching, a scheduler, virtual memory, system calls, and a file system, running on emulated hardware

Course description

An operating system does two things. It **abstracts** hardware into something programmable — a disk becomes a file, a timer becomes a scheduler, physical memory becomes an address space. And it **arbitrates** between programs that would otherwise interfere — deciding who runs, who gets memory, who may read what.

Both jobs are exercises in building a convincing illusion on uncooperative hardware. A process believes it has the whole machine; it has a few milliseconds at a time. It believes it has a contiguous address space; it has scattered physical pages. It believes a write landed on disk; it is in a buffer. Every one of those illusions leaks, and knowing where is most of what systems programming is.

This course is built around implementation. Reading about context switching is not the same as writing the assembly that saves a register set and returns into a different stack. The xv6 teaching kernel — a small, complete, readable Unix — is the vehicle, and by the end you will have modified every major subsystem in it.

CSCE 614 explained the machine. This course explains the software that makes the machine usable by more than one program at a time, and the two are deliberately adjacent: scheduling decisions are cache decisions, and page tables are the hardware you already met.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Explain the kernel/user boundary and how control crosses it.
- 2 Implement context switching and explain exactly what must be saved.
- 3 Compare scheduling policies and reason about the latency/throughput trade.
- 4 Implement locks and condition variables, and explain why each primitive exists.
- 5 Explain deadlock's four conditions and apply the standard responses.
- 6 Implement a page-table-based virtual memory system with demand paging.
- 7 Implement a file system with crash consistency, and explain why journaling works.
- 8 Explain the isolation mechanisms behind containers and virtual machines.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	What an Operating System Is For	What problem does a kernel actually solve?
2	Processes, Threads, and Context Switching	What exactly must be saved to stop one program and start another?
3	CPU Scheduling	How do you schedule jobs whose running times you do not know?
4	Synchronization	How do you let many threads touch the same data safely?
5	Deadlock	Why do correctly-locked programs stop, and what do you do about it?
6	Memory Allocation	Why is malloc hard, and why is it fast anyway?
7	Virtual Memory and Paging Policy	What does the kernel do when a page is not there?
8	File Systems	How do you store a growing file on fixed-size blocks?
9	Crash Consistency	How do you make a multi-block update atomic on a device with no transactions?
10	I/O and Device Drivers	How does a kernel manage devices a million times slower than itself?
11	Protection and Isolation	What mechanisms separate one workload from another, and how strong is each?
12	Distributed Aspects: RPC and Network File Systems	What breaks when a system call crosses a network?
13	Kernel Design	Where should the boundary between kernel and user code be?

Projects

Project 1 — Scheduler and system calls (after Module 5)

Add new system calls and replace xv6's scheduler. The deliverable is a measurable change in behaviour, not just code that compiles.

Requirements

- At least three new system calls, including one that returns information about the calling process.
- A replacement for xv6's round-robin scheduler: implement multi-level feedback queues with aging.
- Priority inheritance on at least one lock, demonstrating that it fixes priority inversion.
- Instrumentation recording per-process run time, wait time, and context switch count.
- A workload mix of CPU-bound and I/O-bound processes used to compare schedulers.

Done means

- A table comparing round-robin and MLFQ on the same workload: average turnaround, average response time, and fairness.
- A demonstration of priority inversion occurring, and then not occurring after inheritance is added.
- A written explanation of one scheduling decision your implementation makes that surprised you.

Project 2 — Virtual memory and a file system (after Module 12)

Implement demand paging with swapping, then make the file system survive crashes. Both must be demonstrated under adversarial conditions, not just happy paths.

Requirements

- Lazy allocation: `sbrk` returns immediately; pages are allocated on first fault.
- Copy-on-write fork, with page reference counting.
- Demand paging from a backing store, with a page replacement policy of your choice, implemented and justified.
- `mmap` and `munmap` for file-backed mappings.
- A journaling layer for the file system, with recovery on boot.
- A crash injection harness that kills the kernel at a random point during a write.

Done means

- COW fork demonstrated: fork a process with a 100 MB heap and show that physical memory use does not double until writes occur.
- A plot of fault count against working-set size for your replacement policy, compared against a second policy.

- One hundred crash-injection runs with the file system verified consistent after every one. Report the number of runs that required recovery.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
MIT 6.1810 / 6.S081 Operating System Engineering — xv6 (free, with labs)	Primary source. A complete small Unix kernel with a book, lecture notes, and graded labs. The single best free OS course in existence.
MIT 6.S081 2020 archive — lecture video and labs	The version with full recorded lectures. Use alongside the current course page.
OSTEP — Operating Systems: Three Easy Pieces (free book)	The best written OS textbook, free from the authors. Organised as virtualisation, concurrency, persistence — which is this course's structure.
xv6: a simple, Unix-like teaching operating system (free book)	The commentary on the xv6 source. Read it with the code open beside it.
Berkeley CS162 (free lectures)	A third voice, with more emphasis on distributed systems and security.
Linux kernel documentation	What a production kernel actually does, for when you want to know how the real thing differs.

Tooling

- **xv6-riscv** and the RISC-V toolchain. RISC-V because the privileged architecture is clean and documented in a readable specification, unlike x86.

- **QEMU** for emulation. You will not brick anything, and you can attach a debugger to a kernel mid-boot.
- **GDB** attached to QEMU. Being able to break inside an interrupt handler is the difference between understanding and guessing.
- **A Linux or WSL environment.** The toolchain assumes it.
- **A portfolio repository** with your kernel at each lab stage, and written notes on what broke.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.